

Obsidian Summarizer

Generative KI für Softwareentwickler

von

Kaan Kaplan, Gian Luca Tehrani, Yannik Gartmann

Bern, 05. Mai 2026

Inhaltsverzeichnis

1	ARCHITECTURAL DECISION RECORD (ADR)	3
1.1	KONTEXT	3
1.2	OPTIONEN	4
1.2.1	Option 1: Lokale Architektur mit lokalem LLM und lokalem RAG	4
1.2.2	Option 2: Cloud-basierte Architektur mit externem LLM	4
1.2.3	Vergleich der Optionen	5
1.3	ENTSCHEIDUNG	5
1.4	UMSETZUNGSANSATZ UND HILFSMITTEL	6
1.5	ARCHITEKTURSKIZZE	6
2	PRODUCT REQUIREMENTS DOCUMENT (PRD)	7
2.1	PRODUKTÜBERSICHT	7
2.2	ZIEL DES PRODUKTES	7
2.3	PROBLEMSTELLUNG	7
2.4	PRODUKTUMFANG	8
2.5	FUNKTIONALE ANFORDERUNGEN	8
2.6	NICHT FUNKTIONALE ANFORDERUNGEN	8
2.7	SICHERHEIT UND DATENSCHUTZ	9
3	ANFORDERUNGSPROFIL UND SPEZIFIKATION (MVP)	9
3.1	EXECUTIVE SUMMARY	9
3.2	USER STORIES & SCOPE (MVP)	10
3.2.1	US-01	10
3.2.2	US-02	10
3.2.3	US-03	10
4	TECHNISCHE ARCHITEKTUR	10
4.1	LLM UND PROVIDER	11
4.2	TECHSTACK	12
5	EVALUATION STRATEGIE	12
5.1	METRIKEN	12
5.2	TESTDATENSATZ	13
6	RISIKOANALYSE	13
7	ROADMAP	14
		1

8	TABELLENVERZEICHNIS.....	16
9	ABBILDUNGSVERZEICHNIS.....	16

1 Architectural Decision Record (ADR)

1.1 Kontext

Im Rahmen des Projekts soll ein Tool entwickelt werden, das Benutzerinnen und Benutzer beim Erstellen von Zusammenfassungen innerhalb von Obsidian unterstützt. Die Grundidee besteht darin, ein Obsidian Plugin zu entwickeln, das Markdown-Dateien aus einem Ordner eines Obsidian Vault analysiert und daraus automatisch eine strukturierte Zusammenfassung erzeugt.

Das Tool soll über die Obsidian-Oberfläche bedienbar sein, beispielsweise über ein Kontextmenü mit der Aktion „Summary erstellen“. Nach dem Start der Aktion liest das Plugin relevante Markdown-Dateien aus dem Vault, bereitet die Inhalte auf, sucht relevante Kontexte über einen Retrieval-Mechanismus und übergibt diese gemeinsam mit einem Prompt an ein lokales Large Language Model. Das Ergebnis wird als Markdown-Datei, zum Beispiel `summary.md`, im gleichen Ordner oder in einem definierten Zielordner gespeichert.

Die Architektur basiert auf folgenden Hauptkomponenten:

- Obsidian UI: Einstiegspunkt für den Benutzer, z. B. über ein Kontextmenü.
- Obsidian Plugin: Zentrale Steuerung des Workflows.
- Markdown-Dateien im Vault: Wissensbasis und Inputdaten.
- Chunking-Komponente: Zerlegt Markdown-Dateien in kleinere Abschnitte.
- Embedding-Erzeugung: Berechnet Vektor-Repräsentationen der Textabschnitte.
- Vektorindex / Retrieval Store: Speichert Embeddings und ermöglicht semantische Suche.
- Lokales LLM über Ollama: Generiert die eigentliche Zusammenfassung.
- `summary.md`: Ausgabe der generierten Zusammenfassung.

Wichtig ist dabei die klare Trennung der Verantwortlichkeiten: Das Plugin erstellt und speichert die Datei. Das LLM erzeugt nur den Inhalt der Zusammenfassung. Dadurch wird der begrenzte Kontext des LLMs nicht mit nicht relevanten Informationen überfüllt und das Plugin verwendet die standardisierten Events von Obsidian, um auf Änderungen im Vault zu reagieren.

1.2 Optionen

Für die Umsetzung des Obsidian Plugins wurden zwei grundsätzliche Architekturvarianten betrachtet. Beide Varianten verfolgen dasselbe Ziel: Markdown-Dateien aus einem Obsidian Vault sollen analysiert und daraus automatisch eine Zusammenfassung erstellt werden. Der Unterschied liegt vor allem darin, wo die eigentliche KI-Verarbeitung stattfindet und welche Abhängigkeiten dadurch entstehen.

1.2.1 Option 1: Lokale Architektur mit lokalem LLM und lokalem RAG

Bei dieser Option findet die gesamte Verarbeitung lokal auf dem Gerät des Benutzers statt. Das Obsidian Plugin liest die Markdown-Dateien aus dem Vault, zerlegt sie in kleinere Abschnitte, erzeugt Embeddings und speichert diese in einem lokalen Vektorindex. Für die Zusammenfassung wird ein lokales LLM, beispielsweise über Ollama, verwendet.

Das Plugin steuert den gesamten Workflow: Es liest die Dateien, bereitet die Inhalte auf, sucht relevante Kontexte über den Vektorindex, erstellt den Prompt, ruft das lokale LLM auf und speichert die generierte Zusammenfassung als Markdown-Datei im gleichen Ordner.

Einer der Hauptaspekte dieser Variante ist, dass alle Daten lokal bleiben. Es müssen keine Inhalte aus dem Obsidian Vault an externe Anbieter übertragen werden. Dies ist besonders relevant, da ein Vault persönliche Notizen, Studienunterlagen oder vertrauliche Projektdokumentationen enthalten sein können. Weiter besteht keine Abhängigkeit von Cloud Anbietern und somit entstehen auch keine laufenden Kosten durch API-Key Usages.

Ein negativer Punkt ist, dass die Qualität und Geschwindigkeit sehr von der lokalen Hardware und vom verwendeten Modell abhängen. Ausserdem muss ein lokales LLM installiert und betrieben werden. Für einen universitären Prototyp ist diese Einschränkung jedoch akzeptabel.

1.2.2 Option 2: Cloud-basierte Architektur mit externem LLM

Bei dieser Option übernimmt das Obsidian Plugin weiterhin die Benutzerinteraktion und Dateiverwaltung. Die eigentliche KI-Verarbeitung wird dann jedoch über einen externen LLM-Anbieter erfolgen. Das Plugin extrahiert und sendet die relevanten Markdown-Inhalte oder Chunks an eine Cloud-API, um die generierte Zusammenfassung anschliessend als Markdown-Datei im Vault speichern.

Cloud basierte Modelle sind häufig leistungsfähiger und es wird keine leistungsfähige lokale Hardware benötigen. Zudem muss der Benutzer kein eigenes Modell lokal installieren.

Diese Variante hat jedoch wesentliche Nachteile. Inhalte aus dem Obsidian Vault müssten an externe Dienste übertragen werden, was Datenschutzfragen und Sicherheitsfragen aufwirft. Zusätzlich können laufende Kosten pro Anfrage oder Token entstehen. Die Lösung wäre ausserdem abhängig von Internetverbindung, API-Verfügbarkeit, Rate Limits, Preisänderungen und Anbieterbedingungen.

1.2.3 Vergleich der Optionen

Kriterium	Option 1: Lokal	Option 2: Cloud
Datenschutz	Sehr gut, da Daten lokal bleiben	Schwächer, da Daten extern verarbeitet werden. Abhängig vom Anbieter
Sicherheit	Hoch, keine externe Übertragung	Abhängig vom Anbieter
Kosten	Keine laufenden API-Kosten	Laufende Token/API-Kosten möglich
Machbarkeit	Gut mit Obsidian Plugin und Ollama	Gut, aber abhängig von API-Key und Internet
Qualität	Abhängig vom lokalen Modell	Häufig besser durch stärkere Modelle
Abhängigkeiten	Gering	Hoch, da externer Anbieter notwendig
Integration in Obsidian	Sehr passend zum lokalen Mark-down-Konzept	Möglich, aber weniger konsequent lokal

Tabelle 1 Vergleich der Varianten

1.3 Entscheidung

Wir entscheiden uns für eine lokal ausführbare Architektur mit einem Obsidian Plugin, lokalem Retrieval-Augmented Generation Ansatz und lokalem LLM über Ollama. Der Ansatz

mit Cloud-Diensten ist für einen universitären Prototyp technisch möglich, passt aber weniger gut zu den Anforderungen bezüglich Datenschutzes, Kostenkontrolle und Unabhängigkeit.

Die Verarbeitung der Markdown-Dateien soll vollständig lokal auf dem Gerät des Benutzers stattfinden. Inhalte aus dem Obsidian Vault werden nicht an externe Cloud-Dienste übertragen. Somit übernimmt das Plugin die Steuerung des gesamten Workflows, es verarbeitet die Obsidian internen Events, erzeugt oder aktualisiert den lokalen Vektorindex, baut den Prompt, ruft das lokale LLM auf und speichert die generierte Zusammenfassung als Markdown-Datei.

1.4 Umsetzungsansatz und Hilfsmittel

Für die Umsetzung wird ein KI-Agent Unterstützen Coding verwendet. Das bedeutet, dass die Entwicklung iterativ und unterstützt durch KI-Werkzeuge wie Cursor oder Claude Code erfolgt. Trotzdem wird die Architektur des Prototyps durch das Projektteam erarbeitet, um eine solide Basis für die KI-Agenten zu schaffen, welche durch die erstellten Anforderungen und Richtlinien einen Leitfaden erhalten. So kann Halluzination und Fehlprogrammierung durch die Agenten entgegengewirkt werden.

1.5 Architekturskizze

Obsidian Plugin Architektur mit RAG und lokalem LLM

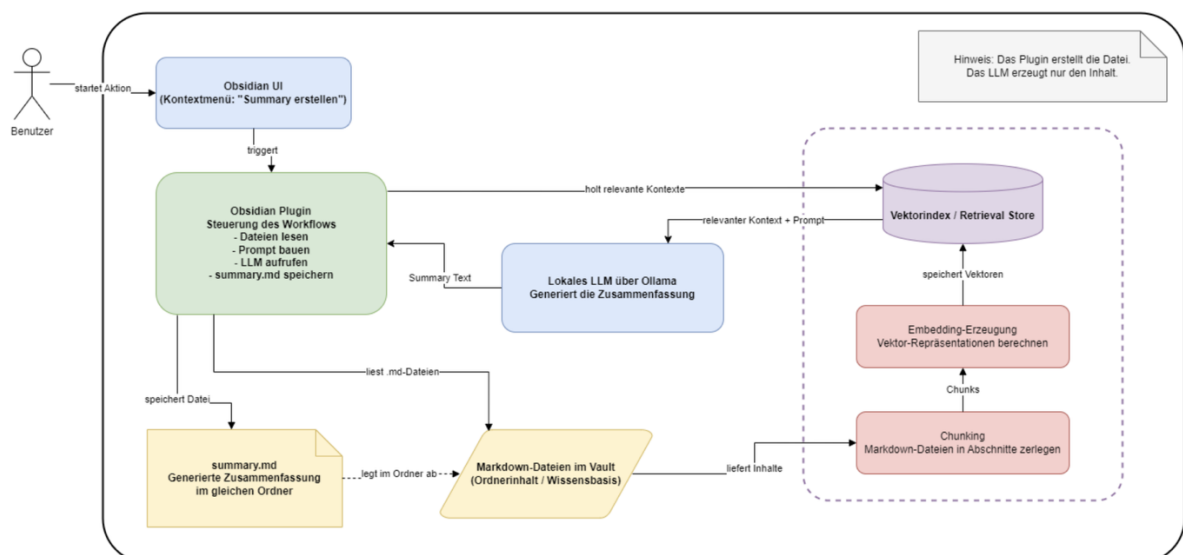


Abbildung 1 Architekturskizze

2 Product Requirements Document (PRD)

2.1 Produktübersicht

Das geplante Produkt ist ein Obsidian Plugin, das aus bestehenden Markdown-Dateien automatisch eine Zusammenfassung erstellt. Die Funktion soll direkt in Obsidian verfügbar sein, beispielsweise über ein Kontextmenü oder einen Command wie „Summary erstellen“.

Die Lösung wird lokal umgesetzt. Das Plugin liest Markdown-Dateien aus dem Obsidian Vault, verarbeitet die Inhalte lokal und nutzt ein lokales LLM, um daraus eine Zusammenfassung zu generieren. Die erzeugte Zusammenfassung wird wieder als Markdown-Datei im Vault gespeichert.

Somit werden keine Inhalte an externe Cloud-Dienste übertragen und somit ist die Lösung gut geeignet für persönliche Notizen, Studienunterlagen oder vertrauliche Projektdokumentationen.

2.2 Ziel des Produktes

Ziel des Plugins ist es, den manuellen Aufwand beim Zusammenfassen von Obsidian-Inhalten zu reduzieren. Benutzer sollen mit wenigen Klicks eine strukturierte Zusammenfassung aus ihren Markdown-Dateien erstellen können.

Wichtige Ziele sind:

- einfache Nutzung direkt in Obsidian
- automatische Erstellung einer Zusammenfassung
- lokale Verarbeitung der Daten
- keine laufenden API-Kosten
- gute Eignung als universitärer Prototyp

2.3 Problemstellung

Obsidian Vaults können mit der Zeit sehr umfangreich werden. Informationen sind oft über mehrere Markdown-Dateien verteilt. Wenn eine Zusammenfassung erstellt werden soll, müssen Benutzer die relevanten Dateien manuell lesen, wichtige Inhalte erkennen und diese selbst zusammenfassen.

Dieser Prozess ist zeitaufwendig und mühsam. Das Plugin soll diesen Aufwand reduzieren, indem es relevante Inhalte aus dem Vault verarbeitet und daraus automatisch eine Zusammenfassung erstellt.

Gleichzeitig müssen Datenschutz, Sicherheit und Kosten berücksichtigt werden. Da Obsidian Vaults sensible oder persönliche Inhalte enthalten können, ist eine lokale Verarbeitung besonders sinnvoll.

2.4 Produktumfang

Der Prototyp soll folgende Kernfunktionalität bieten:

1. Der Benutzer startet das Plugin über die Obsidian-Oberfläche.
2. Das Plugin liest relevante Markdown-Dateien aus dem Vault.
3. Die Inhalte werden lokal verarbeitet.
4. Ein lokales LLM erstellt daraus eine Zusammenfassung.
5. Die Zusammenfassung wird als Markdown-Datei im Vault gespeichert.

Nicht im Fokus stehen Cloud-Synchronisierung, Benutzerkonten, Mehrbenutzerfunktionen oder eine produktionsreife Veröffentlichung im Obsidian Plugin Store.

2.5 Funktionale Anforderungen

ID	Anforderung	Priorität
PRD-F01	Das Plugin muss über die Obsidian-Oberfläche erreichbar sein, z. B. über ein Kontextmenü oder einen Command.	Muss
PRD-F02	Das Plugin muss Markdown-Dateien aus dem Obsidian Vault lesen können.	Muss
PRD-F03	Das Plugin muss aus den gelesenen Markdown-Dateien eine Zusammenfassung erstellen können.	Muss

Tabelle 2 Funktionale Anforderungen

2.6 Nicht funktionale Anforderungen

ID	Anforderung	Beschreibung	Priorität
PRD-NF01	Datenschutz	Alle Inhalte bleiben lokal auf dem Gerät des Benutzers.	Muss
PRD-NF02	Sicherheit	Es werden keine Vault-Inhalte an externe Dienste übertragen.	Muss

PRD-NF03	Kostenkontrolle	Die Lösung funktioniert ohne laufende API-Kosten.	Muss
----------	-----------------	---	------

Tabelle 3 Nicht funktionale Anforderungen

2.7 Sicherheit und Datenschutz

Sicherheit und Datenschutz sind ein wichtiger Bestandteil des Plugins. Obsidian Vaults können persönliche Notizen, Studienunterlagen oder vertrauliche Projektdokumentationen enthalten. Deshalb soll die Verarbeitung nur lokal auf dem System der Benutzerin oder des Benutzers stattfinden.

Das Plugin liest die Markdown-Dateien lokal aus dem Vault, verarbeitet sie lokal und speichert die erzeugte Zusammenfassung lokal. Es ist nicht vorgesehen, Inhalte an externe LLM-Anbieter oder Cloud-APIs zu senden.

Dadurch ergeben sich folgende Vorteile:

- keine Übertragung sensibler Inhalte an Drittanbieter
- keine Speicherung von Daten auf externen Servern
- bessere Kontrolle über den Datenfluss
- geringeres Datenschutzrisiko
- gute Eignung für private und vertrauliche Wissensbestände

Trotzdem sollte das Plugin sorgfältig umgesetzt werden. Es sollte nur auf relevante Dateien innerhalb des Vaults zugreifen und bestehende Dateien nicht unbeabsichtigt überschreiben.

3 Anforderungsprofil und Spezifikation (MVP)

3.1 Executive Summary

Name: Obsidian Summarizer

Der Obsidian Summarizer erstellt per Klick automatisch lokale, KI-generierte Zusammenfassungen aus Markdown-Dateien in einem Obsidian Vault und speichert sie als eigene Zusammenfassung im Vault wieder ab.

Die Zielgruppe sind Obsidian-Nutzerinnen und -Nutzer, die viele Markdown-Notizen verwalten und daraus schnell strukturierte Zusammenfassungen erstellen möchten.

3.2 User Stories & Scope (MVP)

Für das MVP konzentriert sich der Obsidian Summarizer auf die wichtigsten Funktionen, damit Benutzerinnen und Benutzer aus bestehenden Markdown-Dateien schnell eine lokale Zusammenfassung erstellen können.

Nachfolgend wird "Benutzer" als Synonym für Benutzerinnen und Benutzer verwendet

3.2.1 US-01

Als Obsidian-Benutzer möchte ich einen alle Markdown Dateien in einem Ordner per Rechtsklickkontextmenüeintrag zusammenfassen, damit ich nicht jede Datei einzeln lesen muss. Im Kontextmenü eines Ordners ist die Aktion „Create Summary“ verfügbar.

3.2.2 US-02

Als Benutzer möchte ich eine KI Generierte Zusammenfassung haben, die die wichtigsten Punkte kurz, prägnant und inhaltlich korrekt zusammenfasst.

3.2.3 US-03

Als Benutzer möchte ich die generierte Zusammenfassung direkt im Vault speichern, damit sie wie eine normale Obsidian-Notiz weiterverwendet werden kann. Die Datei `summary.md` wird im ausgewählten Ordner erstellt oder aktualisiert

4 Technische Architektur

Die technische Architektur des Obsidian Summarizers ermöglicht ein lokal ausführbares Obsidian Desktop Plugin. Das Plugin wird direkt in Obsidian integriert und erweitert die Oberfläche um genau eine Kontextmenü-Aktion auf Ordnersebene: Create Summary. Wird diese Aktion ausgeführt, durchsucht das Plugin den ausgewählten Ordner sowie alle Unterordner nach Markdown-Dateien, verarbeitet deren Inhalte lokal und erstellt daraus eine Zusammenfassung. Zudem werden in den Optionen ein neuer Menüpunkt erstellt, wo der Benutzer gewisse Anpassungen vornehmen kann.

Bereich	Technologie	Zweck
Benutzeroberfläche	Obsidian Desktop Plugin API	Integration in Obsidian und Bereitstellung der Kontextmenü-Aktion sowie Optionen in den Einstellungen

Programmiersprache / Runtime	TypeScript / JavaScript, Node.js 20+	Entwicklung und Build des Plugins
LLM-Provider	Ollama lokal	Lokale Ausführung des Sprachmodells
Generierungsmodell	<code>gemma4:e2b</code>	Standardmodell für die Zusammenfassung
Optionales Modell	<code>gemma4:e4b</code>	Qualitätsstärkeres Modell für leistungsfähigere Hardware (Soll über die Einstellungen konfiguriert werden können)
Embedding-Modell	<code>nomic-embed-text</code>	Erstellung von Vektor-Repräsentationen der Markdown-Inhalte
Vektordatenbank	SQLite mit <code>sqlite-wasm-vec</code>	Lokale Speicherung und semantische Suche von Embeddings
Persistenz	<code>vectors.db</code>	Lokale SQLite Datenbankdatei im Plugin-Verzeichnis
Ausgabeformat	Markdown	Speicherung der generierten Zusammenfassung als <code>summary.md</code>

4.1 LLM und Provider

Für die Textgenerierung wird ein lokales LLM über Ollama verwendet. Standardmässig nutzt das Plugin das Modell `gemma4:e2b`, da es für ein MVP leichtergewichtig ist und auf durchschnittlicher Hardware besser ausführbar bleibt. Auf leistungsfähigeren Geräten kann optional `gemma4:e4b` über die Einstellungen verwendet werden. Dieses Modell ist ressourcenintensiver und langsamer, kann jedoch qualitativ bessere Zusammenfassungen erzeugen.

Für die Kommunikation mit dem Modell wird der Standard Ollama API Gateway verwendet, welcher unter `localhost:11434` erreichbar ist.

4.2 Techstack

- Obsidian Desktop Plugin API: Integration des Plugins direkt in Obsidian und Bereitstellung der Kontextmenü-Aktion Create Summary.
- TypeScript / JavaScript: Umsetzung der Plugin-Logik.
- Node.js 20+ und npm: Build, Installation und Entwicklung des Plugins.
- Ollama: Lokaler Provider für das LLM und die Embedding-Modelle.
- `gemma4:e2b`: Standardmodell für die Generierung der Zusammenfassung.
- `gemma4:e4b`: Optionales leistungsstärkeres Modell für bessere Qualität auf stärkerer Hardware.
- `nomic-embed-text`: Lokales Embedding-Modell zur Erstellung von Vektor-Repräsentationen der Markdown-Inhalte.
- SQLite: Lokale Datenbank zur Speicherung der Vektordaten.
- `sqlite-wasm-vec`: Erweiterung für Vektorsuche innerhalb der lokalen SQLite-Datenbank.
- Markdown: Eingabe- und Ausgabeformat; gelesen werden `.md`-Dateien, erzeugt wird `summary.md`.

5 Evaluation Strategie

Der MVP wird als abgenommen gekennzeichnet, wenn die Hauptfunktionalität verfügbar ist: Benutzerinnen und Benutzer können Zusammenfassungen über das Kontextmenü auf einem Ordner in Obsidian erstellen.

5.1 Metriken

Die folgenden Metriken werden verwendet, um die Vollständigkeit des MVPs zu messen.

Metrik	Zielwert	Testsetup	Messbarkeit
Generierungszeit	< 80 Sekunden	MacBook M4 Pro mit geringer Systemauslastung.	Die Zeit wird gestoppt von diesem Zeitpunkt, wenn der Kontextmenüeintrag "Create Summary" ausgewählt wird.
Inhaltsabdeckung	80% Schlüsselpunkte sind erwähnt	Testordner mit vorgefertigten Markdown Dateien.	Es werden drei Zusammenfassungen in einen Ordner gefügt, welche Allgemeinwissen beinhaltet mit expliziten Fehlern. Das LLM soll diese Fehler in der Zusammenfassung auch beinhalten, so wird sichergestellt, dass

			der Inhalt der Markdown verwendet wurde und nicht die Trainingsdaten.
Ausgabeformat	Markdown	Testordner mit vorgefertigten Markdown Dateien.	Das Plugin erstellt ein Valides Markdown mit Obsidian konformer Syntax. (z.B. für LaTeX \$ und \$\$)

Tabelle 4 Ziel Metriken & Messbarkeiten

5.2 Testdatensatz

Als Testdatensatz werden von einem beliebigen Agenten (z.B. Cursor oder Claude Code) drei lange Zusammenfassungen generiert mit Allgemeinwissen (respektive sehr verbreitetes Wissen) welche expliziten Fehler eingebaut haben. Als Beispiel könnten eine Liste von Zaubersprüchen, welche in allen Harry Potter Büchern vorkommen, erstellt und dabei zwei Zusätzliche hinzugefügt werden.

6 Risikoanalyse

Da der MVP nur Lokal läuft, wird auf eine ausführliche Risikoanalyse verzichtet. Wobei eine einfache Analyse ergeben hat, dass folgende Punkte mögliche Risiken darstellen.

Risiko	Eintrittswahrscheinlichkeit	Massnahmen
Gewählte Technologie Stack funktioniert nicht	unwahrscheinlich	Ausführliche Planung und Analyse (wird nicht dokumentiert)
Obsidian hebt Plugin Unterstützung auf	sehr unwahrscheinlich	keine
Ollama ist nicht mehr verfügbar	sehr unwahrscheinlich	keine

Die gewählten Modelle von Google sind nicht mehr verfügbar	sehr unwahrscheinlich	Alternativen müssen bei Eintreten des Risikos evaluiert werden. Es benötigt keine Vorabevaluierung.
Komplettausfall des Projektteams	sehr unwahrscheinlich	keine

Tabelle 5 Risikoanalyse

Basierend auf den oben identifizierten Risiken und Eintrittswahrscheinlichkeit stufen wir das nicht Gelingen des MVP auf "sehr gering" ein.

7 Roadmap

Für die Implementierung des MVP wird folgende Roadmap verwendet.

Schritt	Bezeichnung	Beschreibung
1	Projektsetup	Erstellen des Git Repo inklusive Berechtigungen des Projektteams
2	Agent Setup	Erarbeiten passender Skills und Anweisungen für die Agenten (Cursor)
3	Planung	Gemäss dem Anforderungskatalog wird ein Projektplan zusammen mit dem Agenten erstellt, welcher dann von mehreren Agenten erarbeitet werden kann.
4	Minimale Implementation	Es wird einen Agenten beauftragt die erste Implementation vorzunehmen, dies beinhaltet das Setup des Plugins inklusive Boilerplate Code für das weitere Vorgehen.
5	Kommunikation mit dem LLM	Lokal wird Ollama installiert und die Modelle heruntergeladen. Ein Agent implementiert anschliessend die Kommunikation mit Ollama JS und ermöglicht das Generieren von Files. Im Hintergrund wird ein System-Prompt dem LLM mitgegeben, damit eine Konsistente Qualität der Zu-

		sammenfassungen gewährleistet wird. Das Systemprompt wird vorab nicht genau spezifiziert und auf "Try and Error" Basis bei der Implementierung erarbeitet.
6	Einbau RAG	Ein Agent wird beauftragt die RAG-Pipeline zu erarbeiten und in eine Vektorendatenbank basierend auf SQLite einzupflügen. Die Pipeline wird autoamtisch beim Starten von Obsidian ausgeführt und hört auf den Event Bus von Obsidian und pflegt die Datenbank basierend auf den Events.
7	Verknüpfung RAG mit LLM	Sobald die RAG-Pipeline steht, wird ein Agent beauftragt die beiden Komponenten zu verbinden.
8	Finalisierung	In diesem Schritt werden die Optionen in den Einstellungen erstellt, welche dem Benutzer erlauben gewisse Einstellungen vorzunehmen.
9	Review	Agenten können Fehler machen, somit ist u.a. ein automatisches Review mittels Agenten-Skill sowie ein manuelles Review des Codes und des MVP vorgesehen. Hierbei wird der Code angeschaut und das Produkt geklicktested.
10	Dokumentation	Ein Agent wird beauftragt die aktuelle Systemarchitektur und Implementation zu dokumentieren. So wird sichergestellt das zukünftige Erweiterungen durch Agenten mit dem richtigen Kontext erstellt werden. Ein Review der Dokumentation durch das Projektteam ist hier auch vorgesehen.
11	Testen	Gemäss unseren Kriterien aus der Evaluation Strategie (Kapitel 5) werden die Tests durchgeführt und dokumentiert.
12	Freigabe	Nach erfolgreichen Tests ist der MVP freigegeben.

Tabelle 6 Roadmap

8 Tabellenverzeichnis

Tabelle 1 Vergleich der Varianten	5
Tabelle 2 Funktionale Anforderungen	8
Tabelle 3 Nicht funktionale Anforderungen	9
Tabelle 4 Ziel Metriken & Messbarkeiten	13
Tabelle 5 Risikoanalyse	14
Tabelle 6 Roadmap	15

9 Abbildungsverzeichnis

Abbildung 1 Architekturskizze	6
-------------------------------------	---